

Day 2 — Timers, Counters & Troubleshooting

Concept Summary, Student Tasks, and Instructor Solutions

Concept Summary

TON — Timer On-Delay. Q goes true once $ET \geq PT$ while IN stays true continuously. ET resets to 0 the scan IN goes false — it is not held or paused. There is no partial credit: releasing IN at 4.9s of a 5s preset still resets fully to 0.

TOF — Timer Off-Delay. Q goes true immediately on IN's rising edge (zero delay). The delay only applies to the falling edge — Q stays true for PT after IN goes false. This asymmetry (instant on, delayed off) is the most common point of confusion.

RTO — Retentive Timer On. Like TON, but ET is held across IN going false and only cleared by an explicit RES instruction. Used for cumulative time tracking (e.g. total motor run-hours) where individual stop/start cycles should not erase progress.

CTU / CTD — Counters. CTU increments CV once per false→true (rising) edge on CU; Q goes true when $CV \geq PV$; RESET zeroes CV. CTD decrements toward 0 and uses LOAD (not RESET) to set a starting value — the two instructions are not symmetric. Counters are retentive by default (CV holds across CU going false), unlike timers where retentive is the exception (RTO).

R_TRIG / F_TRIG (Edge Detection / ONS). An R_TRIG stores the previous scan's input value and asserts Q true for exactly one scan on a false→true transition. Any rung that should fire 'once per event' (not once per scan the condition is true) needs explicit edge detection — plain contact-to-coil logic has no memory of previous scans.

Forcing I/O. Forcing substitutes a software-set value between the physical terminal and the logic engine, for inputs or outputs, bypassing whatever the logic (including safety interlocks) computed. Used to isolate hardware vs. logic faults and to commission machines in stages. Forces can persist through downloads/mode changes — always clear forces before leaving a system live.

Fault Triage Order. 1) Check CPU state (RUN/STOP/FAULT) and fault log first. 2) Check module-level fault LEDs next — often points to a specific I/O channel or comms issue. 3) Only then move into online logic monitoring. Chasing a logic bug when the real issue is a faulted I/O module wastes time.

State Machines (Traffic Light Project). Combinational logic (Days 1–2 up to this point) depends only on current inputs. A state machine is sequential — output depends on current inputs AND remembered history (current state). Two viable implementations: one INT state register compared with EQU, or three mutually-exclusive BOOL bits set/reset with SET/RES at each transition.

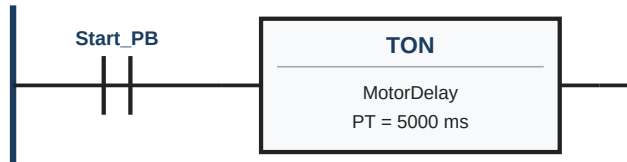
Tasks & Solutions

Organized by Day 2 concept block. Each task is followed immediately by its solution/answer key, with network (rung) diagrams drawn in standard IEC-style symbols, including function-block boxes for timers, counters, and edge-detection instructions.

Task 1 — TON Full-Duration Trace (Timers — TON)

Given: Start_PB is held true for 6 seconds, PT = 5000ms. Sketch the network and state exactly when Q asserts and what ET reads at the moment Start_PB is released.

Solution / Answer Key



Network 1 — Start_PB gates the TON instance MotorDelay.

ET climbs from 0, reaching 5000ms while Start_PB is still true (at the 5s mark), at which point Q goes TRUE. ET keeps climbing past 5000ms while IN remains true (it does not freeze at PT). At the moment Start_PB is released (6s), ET is reset to 0 on the next scan and Q goes false.

Task 2 — TON Early-Release Trace (Timers — TON)

Given: Start_PB is held true for only 3 seconds, PT = 5000ms, then released. What happens to ET and Q? Does it matter that 3s is "most of the way" to the 5s preset?

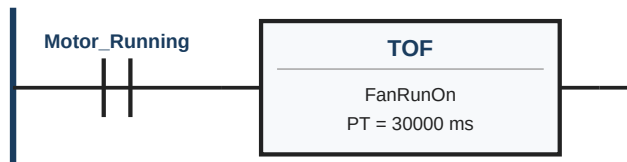
Solution / Answer Key

ET climbs to roughly 3000ms and then, the scan after Start_PB goes false, snaps back to 0 — it is not held or paused at 3000ms. Q never asserts, because ET never reached PT (5000ms). There is no partial credit: this behaves identically whether the release happens at 3s or at 4.9s or a 5s preset.

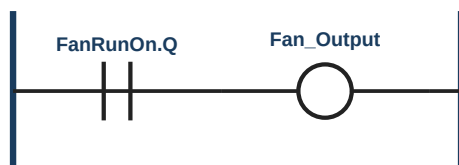
Task 3 — Cooling Fan Build (Timers — TOF)

Build a TOF-based cooling fan circuit: the fan should turn on immediately when the motor starts, and stay on for 30 seconds after the motor stops.

Solution / Answer Key



Network 1 — Motor_Running gates the TOF instance FanRunOn.



Network 2 — FanRunOn.Q drives the physical fan output.

Behavior: Q (and therefore Fan_Output) goes true on the same scan Motor_Running goes true (zero delay on the rising edge). When Motor_Running goes false, Q stays true for the full 30-second PT before finally dropping, keeping the fan running after the motor stops.

Task 4 — Direction Confusion Quiz (Timers — TON vs TOF)

On which edge (rising or falling) does a TOF's Q respond immediately, and on which edge is the delay applied? Contrast this with TON in one sentence.

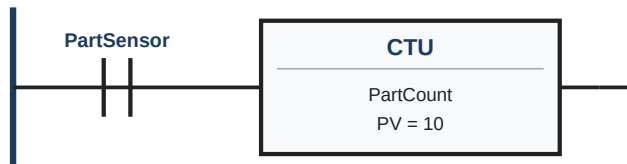
Solution / Answer Key

TOF: Q responds immediately (no delay) on IN's **rising** edge; the delay is applied on the **falling** edge (Q stays true for PT after IN drops). TON is the mirror image — Q is delayed on the way up (must wait for ET to reach PT after IN goes true) and drops immediately, with no delay, the instant IN goes false.

Task 5 — Basic Count Prediction (Counters — CTU)

PartSensor drives CU on a CTU with PV=10, RESET wired to ResetBtn. If PartSensor produces 12 clean rising edges with no reset in between, what do CV and Q read after the 10th edge? After the 12th?

Solution / Answer Key



Network 1 — PartSensor increments the CTU instance PartCount.

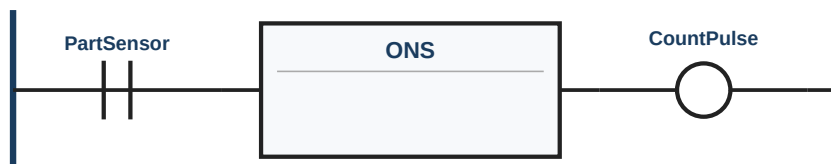
After the 10th edge: CV = 10, Q = TRUE (CV ≥ PV). After the 12th edge: CV = 12, Q remains TRUE — most CTU implementations do not wrap or stop counting once PV is reached; they keep incrementing CV past PV until an explicit RESET or the data type's maximum is hit.

Task 6 — Why Edge Detection Is Needed (Counters / Edge Detection)

A rung increments a stored value every scan that PartSensor is true (not via a counter instruction — a plain ADD-and-store rung). PartSensor is held true for 2 seconds at a ~10ms scan time. Predict roughly how many times the value increments, then explain the fix.

Solution / Answer Key

Prediction: roughly 200 times (2000ms ÷ 10ms ≈ 200 scans), not exactly 200, since scan time is not perfectly fixed. This happens because plain contact-to-coil/ADD logic has no memory of previous scans — it re-executes every single scan the condition is true.



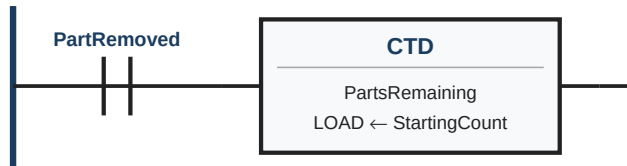
Network 1 — fix: insert a one-shot (ONS / R_TRIG) between the sensor and the increment logic.

With the ONS inserted, CountPulse is true for exactly one scan per physical press, regardless of hold duration — the same 2-second hold now produces exactly 1 increment instead of ~200.

Task 7 — CTU vs. CTD Semantics (Counters — CTD)

Explain in one sentence why calling CTD's PV input a "reset" would be incorrect, contrasting it with CTU's RESET input.

Solution / Answer Key



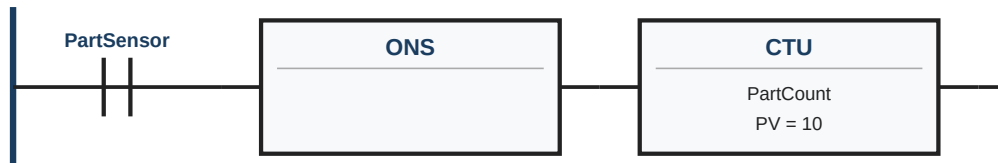
Network 1 — CTD counts down from a loaded starting value rather than resetting to zero.

CTU's RESET input zeroes CV back to 0, but CTD does not "reset" to zero — its LOAD input loads CV to a starting PV value (e.g. a starting inventory count), so the two instructions have different reset semantics and are not symmetric despite looking like mirror-image instructions.

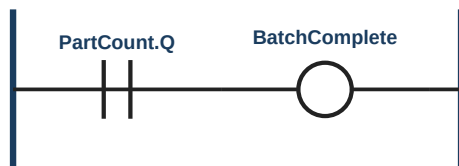
Task 8 — Edge-Triggered Part Counting (Project) (Counters / Edge Detection)

Build: PartSensor → ONS → CTU (PV=10) → Q drives BatchComplete output. ResetBtn drives the counter's RESET. While building, answer: does ResetBtn itself need its own edge detection wired to RESET, or is a held-true reset acceptable?

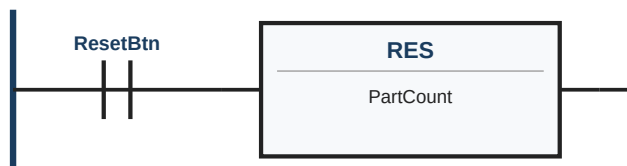
Solution / Answer Key



Network 1 — edge-detected part counting, ONS feeding directly into the CTU's count input.



Network 2 — counter completion drives the BatchComplete output.



Network 3 — ResetBtn clears the counter.

A held-true RESET is acceptable and behaves differently from a held-true CU: for a RESET input, holding true simply holds CV at zero (a steady, correct state), rather than causing repeated miscounting the way a held-true CU would if the counter's own edge-sensing weren't already built in. No extra edge detection is needed on ResetBtn.

Task 9 — Fault Triage Order (Online Monitoring)

List, in the correct order, the three diagnostic steps to take when a machine is misbehaving: (a) online rung monitoring, (b) module fault LEDs, (c) CPU RUN/STOP/FAULT state.

Solution / Answer Key

Correct order: **1) CPU state first** — if it's in FAULT, check the fault log for a specific code before touching logic. **2) Module-level LEDs next** — often points to a specific I/O channel or a backplane communications issue. **3) Only then, online logic monitoring** — inspect rung/instruction behavior. Reasoning: chasing a logic bug for twenty minutes when the real problem is a faulted I/O module is a very common, very avoidable waste of troubleshooting time.

Task 10 — Forcing Exercise (Forcing I/O)

Force PartSensor high without physical actuation and confirm CTU.CV increments. Separately, force Fan_Output on/off directly, independent of FanRunOn.Q. For each forcing action, state which fault category (hardware vs. logic) it would help isolate in a real troubleshooting scenario.

Solution / Answer Key

Forcing PartSensor high and watching CTU.CV increment: if CV increments correctly, the counting *logic* is sound — a real-world failure to count would then point to the physical sensor or its wiring, not the program. This isolates a **hardware/field-wiring** fault from a logic fault.

Forcing Fan_Output directly (bypassing FanRunOn.Q): confirms whether the physical output module and wiring to the fan are sound, independent of whether the TOF logic ever asserts Q. This isolates a **downstream hardware** fault (output module, wiring, actuator) from an **upstream logic** fault.

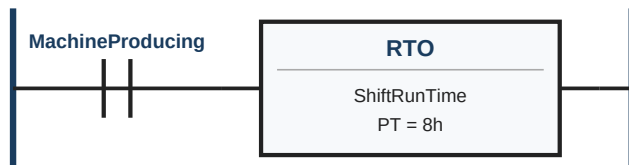
Closing step (required): locate and clear both forces before ending the exercise — a forgotten active force left over from testing is a genuine, recurring real-world hazard.

Task 11 — Shift Accumulation Design (RTO)

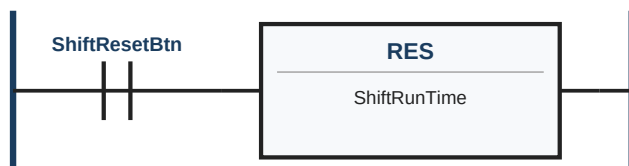
A shift report needs total "machine actively producing" time across an 8-hour shift, with several short stops for material changeovers. Explain why a plain TON is unusable here, then draw the correct instruction.

Solution / Answer Key

A plain TON is unusable because its ET is cleared to 0 every time IN goes false — each changeover stop would discard all previously accumulated time, making an 8-hour cumulative total impossible to obtain from a single TON instance.



Network 1 — RTO accumulates ET across multiple ON/OFF cycles of MachineProducing.



Network 2 — explicit RES instruction required to clear the retentive accumulator at shift end.

Because RTO's ET is held across IN going false, each changeover stop simply pauses accumulation rather than erasing it — the value only clears when the explicit RES instruction fires (e.g. at shift changeover).

Task 12 — Three Planted Faults (Live Debugging)

Using the .ACC/ET monitor first (numeric-value-first troubleshooting), diagnose each planted fault: (1) an RTO's RES rung was deleted; (2) an ONS was removed from in front of a CTU wired to a held input; (3) an RTO was used in a delay-before-alarm rung that should reset cleanly every time.

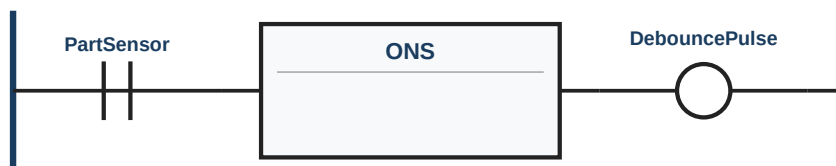
Solution / Answer Key

- (1) **Missing RES on RTO:** ET keeps climbing indefinitely across every ON/OFF cycle and never returns to 0 — checking the .ACC value shows it growing without ever resetting, confirming the RES instruction is missing or unwired.
- (2) **Missing ONS in front of CTU on a held input:** CV increments far faster than expected — one count per scan the input is held true rather than one count per physical event. Comparing CV against the number of intended physical triggers exposes the mismatch immediately.
- (3) **RTO used where TON was needed:** the alarm delay rung keeps stale elapsed time from a previous, unrelated trigger because nothing ever issues a RES — the .ACC value doesn't start from 0 at the start of a new alarm condition, causing the alarm to fire earlier than intended on subsequent triggers. This shows retentive behavior is not universally "better" — using RTO where a self-resetting TON was actually needed creates its own bug class.

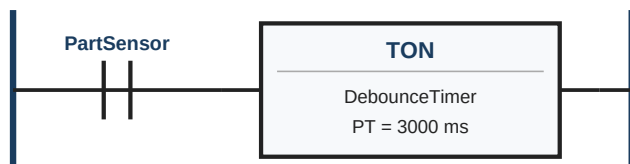
Task 13 — Debounce Counter (Project) (Timer + Counter Combination)

Build a counter that only counts a part after the sensor has been continuously true for a stable 3-second window (debounce before count), rather than counting on a single pulse.

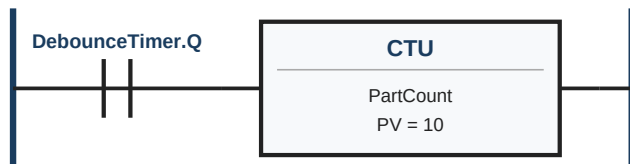
Solution / Answer Key



Network 1 — one-shot marks the initial detection edge.



Network 2 — TON gated directly by PartSensor requires the signal to stay true continuously for the full 3-second window.



Network 3 — only a fully stable 3-second signal reaches the counter.

Key distinction to articulate: this is a different composition than Task 8's pattern. There, ONS fed the counter directly (count once per event). Here, the TON's completion — a condition sustained for the full preset — feeds the counter, so a single glitch shorter than 3 seconds never reaches CTU at all.

Task 14 — State Diagram Annotation (Day Project — State Machine Design)

Sketch the Red → Green → Yellow → Red cycle. For each state, annotate exactly which output is active and which timer is running, before writing any code.

Solution / Answer Key

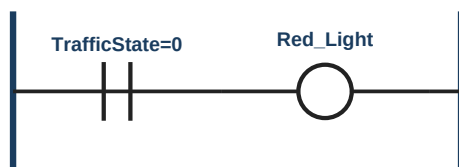
State	Output Active	Timer Running	Preset (PT)
Red	Red_Light	TON_Red	10,000 ms
Green	Green_Light	TON_Green	8,000 ms
Yellow	Yellow_Light	TON_Yellow	3,000 ms

This forces an explicit mapping between the abstract idea of "state" and concrete PLC tags before any code is written — most first-time state-machine design confusion originates from skipping this step.

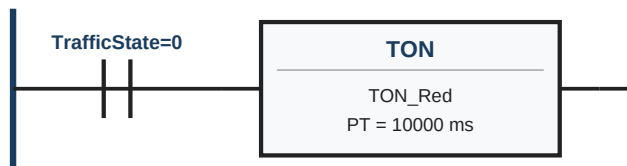
Task 15 — Traffic Light — Option A (INT State + EQU) (Day Project — State Machine Build)

Build the Red state's rungs using one INT state register (TrafficState) compared with EQU instructions. The Green and Yellow states follow the identical pattern with their own preset values.

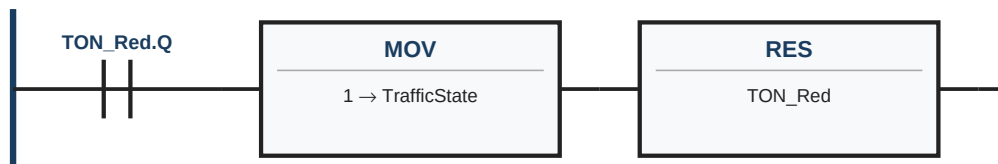
Solution / Answer Key



Network 1 — Red_Light is active only while TrafficState equals 0.



Network 2 — TON_Red only runs while in the Red state.



Network 3 — on timer completion, advance the state and reset the timer for next time around. MOV and RES are output instructions drawn in series on the same rung; both execute together whenever the rung is true.

(Green and Yellow states repeat this exact 3-network pattern): TrafficState=1 -> Green_Light | TrafficState=1 -> TON_Green (PT=8000ms) | TON_Green.Q -> MOV 2,TrafficState + RES TON_Green
TrafficState=2 -> Yellow_Light | TrafficState=2 -> TON_Yellow (PT=3000ms) | TON_Yellow.Q -> MOV 0,TrafficState + RES TON_Yellow

The explicit RES on each timer at the moment of state transition is the exact mechanism that prevents stale ET values from causing a short first pulse the next time that state becomes active again.

Task 16 — Missing RES Bug (Day Project — Debugging)

A student's Red→Green transition rung is missing its RES TON_Red instruction. Using the .ACC/ET monitor, what should the student check, and what symptom would confirm this specific bug?

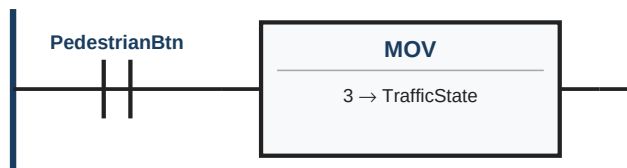
Solution / Answer Key

Check: watch TON_Red.ET at the exact scan TrafficState returns to 0 (i.e., the next time the light cycles back to Red). It should read exactly 0 at that instant. Symptom confirming the bug: on many platforms a TON's ET auto-resets once IN goes false regardless of a missing RES, so this bug may not be visible every cycle — but if the same timer instance is reused with any overlap across the transition scan, a stale ET value can cause a short first pulse (Red_Light staying on briefly less than the full 10s) on the next pass through the loop. The fix is always to include the explicit RES rather than relying on implicit auto-reset behavior, since that behavior is platform-dependent.

Task 17 — Pedestrian Interrupt Priority (Day Project — Stretch)

Add a PedestrianBtn input that forces TrafficState to an all-red + walk-signal state, then returns to Red after its own timer. Should a pedestrian request interrupt an in-progress Green phase immediately, or wait for the current phase to complete naturally?

Solution / Answer Key



Network 0 — placed at the TOP of the program, ahead of the normal state-transition rungs, so the priority check is evaluated before the rest of the logic each scan.

There is no single correct answer — both are defensible: **immediate interrupt** favors pedestrian safety response time but can abruptly change the light for approaching traffic, which may be confusing or feel unsafe from the driver's side. **Latching the request** (acting only once the current phase completes naturally) avoids an abrupt change but delays the pedestrian's crossing. Surfacing this ambiguity is itself the point — it is a first exposure to priority/override design, where requirements outside the ladder logic itself (safety policy, traffic engineering norms) determine the correct choice, not the code.